

Représentations en grille et en graphe pour l'apprentissage par auto-jeu à Hive

Une comparaison pré-enregistrée sous budget de calcul limité

Mohamed Bechir Kefi

Rapport de recherche indépendant (non évalué par les pairs); Version v2.0-draft, Octobre 2026

Résumé

Hive est un jeu de stratégie hexagonal sans plateau dont les coups sont des paires (pièce, destination) sur un ensemble de cellules en perpétuel changement, ce qui en fait, en principe, un candidat naturel pour des encodages par réseaux de neurones en graphe plutôt que pour les encodages convolutifs en grille standards des systèmes de type AlphaZero. Nous testons cette intuition sous un protocole pré-enregistré, gelé avant toute exécution comparative : un CNN en grille et un réseau relationnel à passage de messages de capacité appariée (+1.5%) partagent un moteur validé côté règles, un décodeur d'actions unique, des réglages d'entraînement identiques, une population gelée de trois adversaires et 250 ouvertures gelées, et sont évalués sous deux lectures budgétaires (à exemples d'entraînement égaux ; à temps mural égal à un seuil pré-enregistré), avec cinq graines indépendantes par bras et la troncature rapportée comme un résultat à part entière. L'hypothèse est rejetée : le bras graphe obtient un score inférieur contre deux des trois adversaires sous les deux lectures, à un coût en temps mural double, son déficit se concentrant dans la conversion des positions gagnées ; la plus grande borne supérieure de tout intervalle sur un avantage du graphe est +0.028. Les ablations montrent que les relations d'arêtes typées du bras graphe sont nécessaires à la stabilité de l'optimisation, tandis que son pooling global est dispensable. Le résultat est négatif, pré-enregistré et reproductible à partir des enregistrements publiés.

1 Introduction

Depuis AlphaZero, apprendre à jouer à un jeu de plateau sans exemples humains suit une recette stable : un réseau estime une politique et une valeur pour chaque position, une recherche arborescente guidée par le réseau produit des décisions plus fortes que le réseau seul, et les parties que la recherche joue contre elle-même deviennent les données d'entraînement du réseau suivant (Silver et al., 2018). Un ingrédient de la recette n'est pas général : la manière dont une position est présentée au réseau. Pour les échecs, le shogi et le Go, le plateau est un tableau fixe, une position devient une pile de plans semblables à une image, et un réseau convolutif les lit ; l'encodage en grille fonctionne parce que chaque case existe à chaque instant, à une place fixe.

Hive ne possède pas cette propriété. Il se joue avec des tuiles hexagonales et sans plateau : les tuiles forment la surface de jeu, cette surface change de forme à chaque tour, les tuiles grimpent les unes sur les autres, et un coup se décrit au mieux par « cette pièce va vers cette cellule » sur un ensemble de cellules qui n'existe que relativement à la ruche courante. L'objet naturel pour une telle position est un graphe, avec les cellules comme nœuds, l'adjacence comme arêtes et les piles comme attributs de nœud, et le réseau naturel est un réseau de neurones en graphe, qui n'a besoin d'aucun cadre. Les encodages en grille peuvent néanmoins s'appliquer en dépliant la ruche dans un grand cadre fixe, au prix de choix d'ancrage, d'espace vide et d'un très grand espace de coups discret dont le réseau doit apprendre la structure à partir de zéro.

Il est tentant de conclure qu'un encodage en graphe doit mieux apprendre Hive. Nous avons conçu cette étude parce que l'intuition coupe dans les deux sens. Un réseau en graphe épouse la structure native du jeu et ne porte aucun artefact d'ancrage. Mais l'issue d'une partie de Hive se joue sur des tactiques d'encercllement à courte portée (une abeille reine perd lorsque ses six cellules voisines sont occupées), précisément le régime où la localité convolutive est la plus forte. De plus, un graphe sur les seules cellules occupées ne peut exprimer les destinations légales d'un coup, qui sont des propriétés de l'espace vide ; le graphe doit donc porter des cellules candidates vides, et l'on ne sait pas si un réseau simple à passage de messages apprend les contraintes de glissement, d'escalade et de connexité à partir d'un tel graphe mieux qu'un réseau convolutif ne les apprend à partir du cadre déplié.

Les résultats antérieurs ne tranchent pas la question. Les deux comparaisons directes grille-contre-graphe menées sur des jeux de plateau pointent dans des directions opposées : à Hex, sous un apprenant fondé sur la valeur, un réseau en graphe a dominé sur les dépendances à longue portée tandis que le réseau convolutif restait plus affûté sur les motifs locaux (Keller et al., 2023), et aux échecs, sous auto-jeu, un réseau à attention sur graphe a surpassé les lignes de base convolutives, à partir d'une seule exécution d'entraînement par modèle (Rigaux et Kashima, 2024). La seule étude de type AlphaZero sur Hive a comparé cinq encodages de plateau, tous en grille (de Goede et al., 2022), et aucun de ces travaux n'a fait tourner les deux bras sous un même pipeline d'auto-jeu, à capacité appariée, sous un budget apparié, avec plusieurs exécutions indépendantes par bras, sur Hive.

Notre question principale (RQ-H1) est de savoir si, à budget d'entraînement comparable, une architecture en graphe apprend une meilleure politique qu'une architecture en grille pour Hive en jeu de base, au sein d'un pipeline d'auto-jeu de type AlphaZero. L'hypothèse testée, H1, énonce qu'un réseau en graphe simple à passage de messages, recevant la ruche sous forme de graphe, atteint un score moyen plus élevé contre une population d'adversaires fixe qu'un réseau convolutif en grille recevant un cadre déplié de 32×32 , à budget d'entraînement égal. Deux questions secondaires demandent si la réponse change lorsque le budget est lu comme un nombre d'exemples d'entraînement ou comme un temps mural (RQ-H2), et quels composants de l'architecture en graphe portent son comportement (RQ-H3). Le protocole, gelé avant toute exécution comparative, rejette H1 si, sous les deux lectures de budget, le bras graphe ne montre aucun avantage cohérent entre graines et si l'intervalle au niveau des graines sur la différence de score graphe-grille exclut un avantage graphe substantiel ; il énonce qu'un résultat négatif ou nul est une issue publiable.

Nous revendiquons trois contributions.

- La première comparaison grille-contre-graphe contrôlée, à budget apparié et multi-graines pour Hive, les deux bras étant placés sous le même pipeline de type AlphaZero, lus sous deux égalisations de budget, avec une réponse négative pré-enregistrée.
- Un harnais de comparaison reproductible pour les jeux sans cadre et à empilement : un moteur de règles validé indépendamment de l'apprentissage, deux encodeurs épinglés à l'octet près entre deux langages d'implémentation, un décodeur partagé sur un ensemble d'actions variable, une évaluation consciente de la troncature et une discipline d'artefacts gelés.
- Une attribution par composant pour le bras graphe, dans laquelle les relations typées déterminent l'entraînabilité et le pooling a un effet nul, avec un supplément à deux composants honnêtement étiqueté.

Le rapport technique qui accompagne cet article contient les architectures complètes, les tables brutes par graine, les corpus annotés à la main et la provenance de chaque nombre.

2 Contexte et travaux connexes

Hive. Hive (Yianni, 2010) est un jeu à deux joueurs, à information parfaite et à somme nulle, joué sans plateau. Chaque joueur possède onze tuiles dans le jeu de base : une abeille reine, deux araignées, deux scarabées, trois sauterelles et trois fourmis soldats. Les tuiles sont posées bord contre bord, une tuile entrant en jeu ne peut toucher une tuile adverse, et la reine doit entrer en jeu au plus tard au quatrième tour de chaque joueur. Chaque type d'insecte se déplace différemment (en glissant, en grimant ou en sautant), la ruche doit rester connexe à tout moment, et une tuile qui glisse doit physiquement passer entre ses voisines. Un joueur perd lorsque sa reine est entourée sur ses six côtés. Quatre propriétés rendent Hive malcommode pour les méthodes développées sur des plateaux fixes : la géométrie est sans cadre, les tuiles s'empilent, le facteur de branchement se stabilise autour de 60 (environ le double de celui des échecs) avec des parties qui durent des centaines de plis, et la légalité est non locale, puisque l'existence d'un coup dépend des points d'articulation de la ruche et de l'espace vide (Kampert et al., 2021).

Auto-jeu de type AlphaZero. Un unique réseau associe à un état une politique sur les actions et une valeur pour le joueur au trait ; le réseau guide une recherche arborescente PUCT, et la recherche engendre les parties sur lesquelles le réseau est ensuite entraîné, avec la distribution de visites à la racine comme cible de politique et l'issue de la partie comme cible de valeur (Silver et al., 2018). Des économies que Wu (2020) a introduites dans KataGo, nous en adoptons une, la randomisation du plafond de simulations, parce qu'elle est agnostique à la représentation : la plupart des décisions sont recherchées à bas coût et non enregistrées, et une minorité reçoit une recherche complète et devient un exemple d'entraînement. Jones (2021) a mesuré que le calcul d'entraînement et le calcul de recherche s'arbitrent l'un contre l'autre, de sorte qu'une évaluation doit fixer le budget de recherche à l'identique entre les bras. Agarwal et al. (2021) ont montré que les résultats d'apprentissage par renforcement profond issus d'une poignée d'exécutions d'entraînement sont couramment sur-interprétés, que les comparaisons à 3–5 exécutions ne sont souvent pas soutenues par leurs propres données, et que l'incertitude est

gravement sous-estimée en deçà d’une dizaine d’exécutions ; ils prescrivent des estimations par intervalles issues d’un bootstrap sur les exécutions, que nous adoptons avec la graine comme unité.

Plans en grille contre graphes de cellules. L’encodage canonique de la famille AlphaZero est une pile de plans sur le tableau du plateau, lue par un réseau convolutif résiduel, qu’il faut pour Hive fabriquer en ancrant la ruche dans un cadre. L’alternative remet au réseau la structure d’adjacence elle-même : dans la famille inductive à passage de messages de Hamilton et al. (2017), des poids par couche partagés entre tous les nœuds plongent n’importe quel graphe, y compris des graphes jamais vus à l’entraînement, et chaque position de Hive est un graphe inédit. L’agrégation de voisinage ordinaire ignore la sémantique des arêtes, alors qu’à Hive la direction d’un voisin compte (une sauterelle saute le long d’une ligne ; une porte est formée par les deux cellules flanquant une direction), de sorte que notre bras graphe utilise des relations typées par direction. Un encodage en graphe n’accorde aucune invariance gratuitement : le passage de messages avec des arêtes typées n’est pas invariant par rotation, les champs réceptifs sont limités par la profondeur, et aucune règle de Hive n’est connue du réseau. Les réseaux pointeurs (Vinyals et al., 2015) justifient la notation directe d’un ensemble variable de candidats, le mécanisme derrière notre décodeur d’actions partagé.

Précédents directs et standard de preuve. Trois travaux comparent un encodeur en grille à un encodeur en graphe sur un jeu de plateau ; Tableau 1 les place, avec la seule étude de type AlphaZero sur Hive, à côté du nôtre. Keller et al. (2023) ont mené une comparaison à paramètres appariés sur Hex ($\approx 487K$ contre $\approx 481K$ paramètres, $\approx 110 A100$ -heures par modèle) : le réseau en graphe a commis 1 erreur sur une suite de dépendances à longue portée là où les CNN en commettaient 31 et 36, tandis que le CNN restait plus affûté sur les motifs locaux ; mais la comparaison s’est déroulée sous RainbowDQN, le bras CNN ne s’est jamais entraîné sous recherche arborescente, et le graphe est une réduction propre à Hex dans laquelle chaque nœud est une cellule vide. Rigaux et Kashima (2024) rapportent le signe opposé pour les échecs sous auto-jeu, chaque modèle étant entraîné une seule fois, des « paramètres comparables » allant de 1.0M à 2.2M, et une paramétrisation de l’action qui change avec la représentation. AZ-Hive (de Goede et al., 2022) a croisé cinq encodages denses sur treillis hexagonal avec deux encodages d’action sur un cadre 26×26 , chacun entraîné pendant 4 h avec 5 répétitions, a constaté que l’encodage modifie mesurablement l’apprentissage initial, et ne comportait aucune représentation en graphe dans son espace de conception. Ensemble, ces travaux rendent un avantage du graphe plausible tout en montrant qu’un CNN peut l’emporter sur les motifs locaux, que le résultat positif publié repose sur une exécution par modèle avec un décodeur confondu, et que ce que les arêtes encodent peut compter davantage que la structure de graphe elle-même. Le jeu de Hive est dominé par des tactiques d’adjacence locale autour de la reine, là où les données de Hex favorisent la grille.

Tableau 1 : Les trois précédents directs grille-contre-graphe et la seule étude antérieure de type AlphaZero sur Hive, à côté de la présente conception. « Exécutions par modèle » est le nombre d’exécutions d’entraînement indépendantes derrière chaque comparaison rapportée.

Travail	Jeu	Représentations comparées	Exécutions par modèle	Limite pour la question de Hive
Keller et al. (2023)	Hex	SAGEConv à 15 couches ($\approx 487K$) contre ResNet et U-Net ($\approx 481K$)	1	Comparaison sous RainbowDQN ; CNN jamais sous recherche en auto-

Travail	Jeu	Représentations comparées	Exécutions par modèle	Limite pour la question de Hive
				jeu ; graphe propre à Hex avec actions par nœud
Rigaux et Kashima (2024)	Échecs	Attention sur graphe à caractéristiques d'arêtes (1.0M) contre plans CNN (2.2M)	1	Pas de variance entre graines ; appariement de capacité lâche ; le décodeur change avec la représentation
Ben-Assayag et El-Yaniv (2021)	Othello, Gomoku, Go	GIN à 3 couches sur le treillis carré contre CNN d'AlphaZero	5	Affirmation de transfert sous budgets asymétriques ; actions sur les seuls nœuds
AZ-Hive (de Goede et al., 2022)	Hive, jeu de base	5 encodages en grille $\times$ 2 encodages d'action, tous CNN, cadre 26×26	5	Pas de bras graphe ; conclusions sur la vitesse d'apprentissage initiale
Cette étude	Hive, jeu de base	CNN résiduel sur un cadre 32×32 (1.44 M) contre passage de messages typé par direction sur le graphe de cellules (1.47 M) ; décodeur partagé	5	Une variante, une capacité, une machine, petit budget

3 Système

Un moteur validé indépendamment de l'apprentissage. Chaque bras, chaque adversaire et chaque évaluation partagent un seul moteur de règles, écrit en Rust en juillet 2026 comme projet de moteur antérieur à l'étude. Parce qu'un défaut de règles enseignerait aux deux bras le même jeu faux et invaliderait silencieusement la comparaison, nous avons rétabli sa validation sur la machine d'étude le 9 septembre 2026, avant tout travail sur les adversaires de base ou l'entraînement, selon cinq lignes de preuve. Les comptes perft reproduisent les tables publiées du moteur de référence du protocole (Mzinga) jusqu'à la profondeur 6 pour les huit types de partie de l'Universal Hive Protocol, l'exécution à la profondeur 6 s'achevant en 8.32 s. Le harnais de conformité d'un second moteur de référence (nokamute) a passé 21/21, et le fuzzing différentiel a affirmé l'égalité ensembliste des ensembles de coups légaux avec les deux moteurs de référence après chaque pli sur 27 829 positions. Un corpus de 30 positions critiques annotées à la main à partir des règles de l'éditeur (Yianni, 2010 ; FAQ des règles des World Hive Tournaments) et consigné par commit avant toute exécution du moteur a renvoyé 29 succès et une erreur de mise en place à la première exécution, résolue contre le corpus, puis 30/30 à la seconde ; une suite tactique de cinq positions a été résolue 5/5 par la recherche sans réseau à 400, 1 600 et 6 400 simulations. Des parties aléatoires à graine fixée sur l'ensemble des huit types de partie ont appliqué et annulé 10.9 millions de transitions sans violation d'invariant. Enfin, chaque encodeur existe deux fois, en Rust dans la recherche et en Python dans la boucle d'entraînement, et les deux sont épinglés à l'octet près par des vérifications croisées sur fichiers de référence (240 positions pour l'encodeur grille, 160 sur 160 pour l'encodeur graphe). Rien de tout cela ne prouve la perfection sur parties complètes ; cela borne cinq modes de défaillance différents.

Le pipeline partagé. Les deux bras sont entraînés et évalués par une seule boucle de type AlphaZero avec randomisation du plafond de simulations. Une exécution comprend 10 générations de 500 parties d'auto-jeu sur une seule machine avec quatre threads de travail. L'auto-jeu recherche 128 simulations sur un quart aléatoire des décisions et 32 sur le reste, et seules les décisions à plein budget deviennent des enregistrements ; un bruit de Dirichlet avec $\varepsilon = 0.25$ est mélangé à l'a priori de la racine, les coups sont échantillonnés selon la distribution de visites pendant les 12 premiers plis, un camp abandonne sous -0.92 (l'abandon est désactivé dans 10% des parties à titre d'audit), et une partie qui atteint 300 plis est tronquée, quatrième issue portée à travers le format d'enregistrement, la perte (les enregistrements tronqués ne contribuent à aucune cible de valeur), l'arène et chaque table. L'entraînement est identique pour les deux bras : descente de gradient stochastique au taux d'apprentissage 0.02 sous un calendrier cosinus, momentum 0.9, décroissance des poids 10^{-4} , lot 256, deux époques par génération, et une perte qui ajoute l'entropie croisée de politique sur exactement l'ensemble légal à 0.6 fois l'entropie croisée à trois classes victoire/nulle/défaite sur les enregistrements non tronqués. Une propriété de la boucle doit être énoncée clairement : le réseau de la génération g est une initialisation fraîche à graine fixée, entraînée sur les seuls enregistrements de la génération g , lesquels ont été produits par la recherche guidée par le réseau de la génération $g - 1$; le pilote de campagne ne transmet à l'entraîneur ni point de contrôle de démarrage à chaud ni fenêtre de rejeu. La boucle s'améliore néanmoins de génération en génération, parce que la recherche est plus forte que le réseau brut, mais aucun poids n'est reporté et aucun enregistrement n'est réutilisé. Sept vérifications automatisées du chemin de données et de la boucle d'entraînement (de la normalisation sur l'ensemble légal, en passant par une vraie passe avant, jusqu'à l'assurance que l'évaluation n'alimente jamais l'entraînement) ont été exécutées contre des segments d'auto-jeu réels avant que toute sortie d'entraînement ne soit tenue pour fiable, et un pilote d'une génération a validé la boucle de bout en bout le 10 septembre 2026. Aucun des deux bras n'a reçu de recherche d'hyperparamètres.

Deux représentations, un décodeur. Fig. 1 montre une même position sous les deux encodages et Fig. 2 les deux réseaux. Le bras grille déplie le plateau du moteur par un parcours en largeur, centre la boîte englobante occupée dans un cadre 32×32 , et écrit 77 plans : des plans de pièces indexés conjointement par propriétaire, type d'insecte et niveau de pile, des plans pour les tuiles clouées, la dernière tuile déplacée et les régions de placement des deux camps, et des plans constants pour le camp au trait, les libertés des reines, le pli et les réserves. HiveNet, le réseau grille, est un réseau convolutif résiduel de 96 canaux et 8 blocs résiduels avec un biais de pooling global dans les blocs 2 et 5, une tête de politique de 28 673 logits (28 plans d'emplacements de pièces sur le cadre, plus la passe) et une tête de valeur à trois classes ; il compte 1.44 M paramètres. Le bras graphe est sans coordonnées. Ses nœuds sont les cellules de l'ensemble candidat, chaque cellule occupée et chaque cellule vide adjacente à la ruche, de sorte que chaque destination légale est un nœud ; chaque nœud porte 56 caractéristiques décrivant sa pile niveau par niveau, et un vecteur global de 23 caractéristiques est diffusé à chaque nœud. Les arêtes sont les adjacences dirigées entre cellules candidates, typées par les six directions hexagonales. HiveGraphNet fait passer les caractéristiques de nœud et globales à 152 canaux et applique 8 couches relationnelles à passage de messages, chacune mettant à jour un nœud à partir de lui-même et de ses six voisins directionnels à travers six matrices de poids typées par direction, avec un biais de pooling global (moyenne et maximum masqués des plongements de nœuds) une couche sur trois ; la tête de politique note chaque

paire légale (pièce, destination) à partir du plongement du nœud de destination, du plongement de source de la pièce qui se déplace et d'un plongement d'emplacement ; la tête de valeur lit un pooling masqué moyenne-et-maximum. Il compte 1.47 M paramètres, soit +1.5% par rapport au réseau grille. Les deux bras alimentent un seul décodeur d'actions : les logits n'existent que pour les paires légales (pièce, destination) que le moteur génère, la softmax s'exécute sur exactement cet ensemble, la cible de politique est la même distribution de visites à la racine, et le départage est identique. Aucun des deux bras ne s'entraîne avec augmentation par symétries.

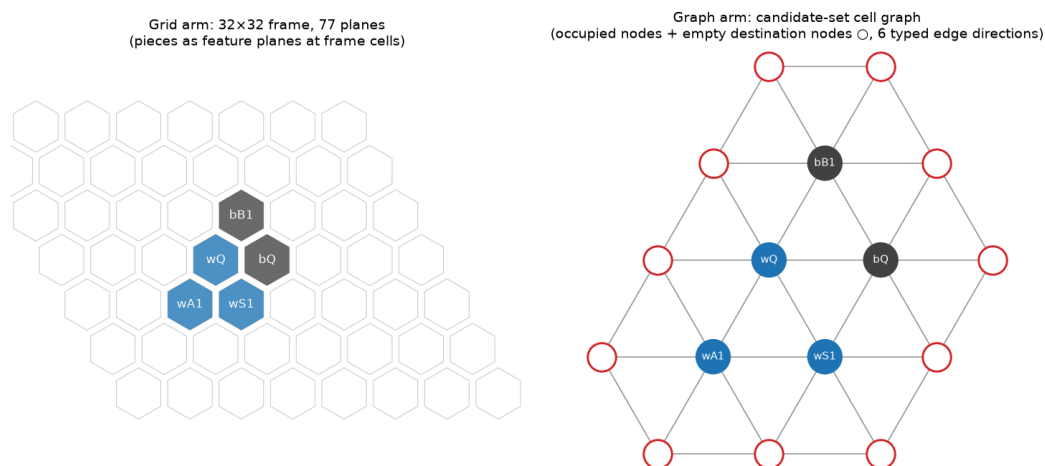


Fig. 1 : Une même position sous les deux encodages. À gauche, la vue du bras grille : la position plongée dans un cadre fixe 32×32 de cellules hexagonales, chaque cellule du cadre portant 77 valeurs de caractéristiques. À droite, la vue du bras graphe : le graphe de cellules sur l'ensemble candidat, dans lequel les cellules occupées (pleines) et les cellules vides adjacentes à la ruche (cercles ouverts) sont reliées par des arêtes typées par les six directions hexagonales. Les deux vues dérivent du même état du moteur sans perte.

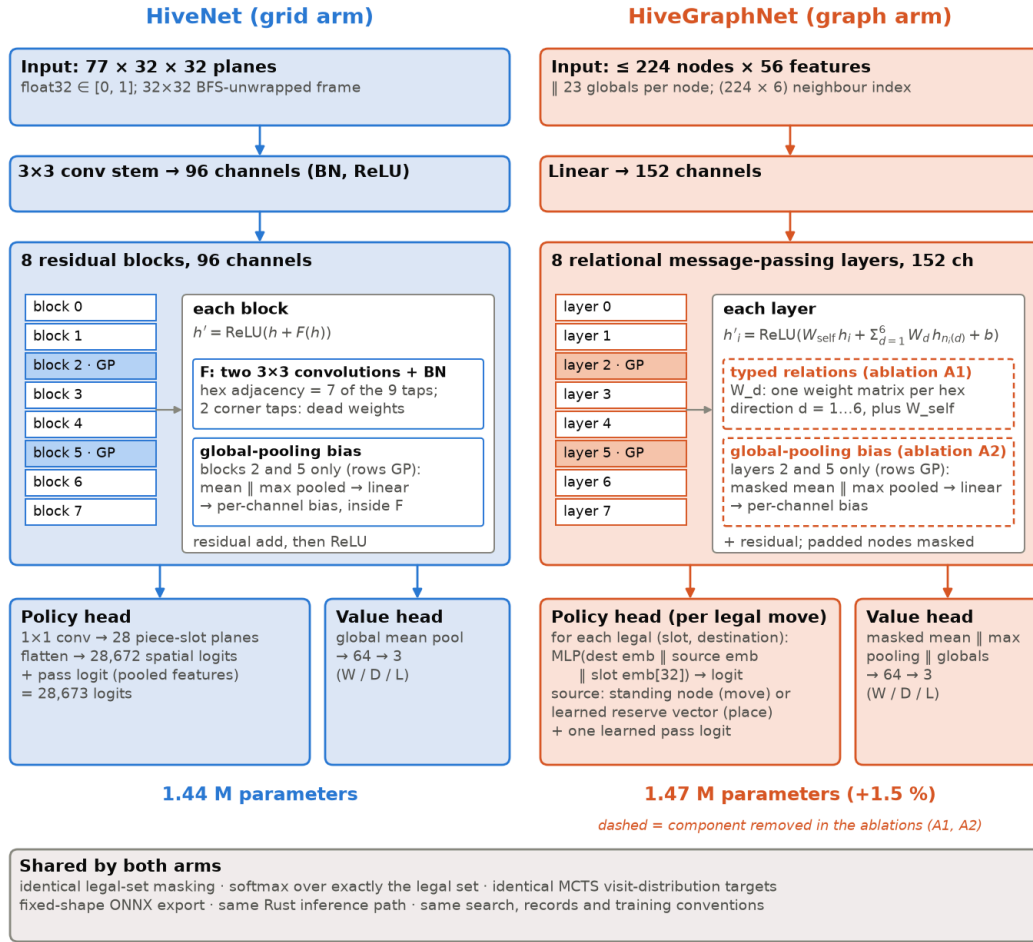


Fig. 2 : Les deux réseaux. À gauche, HiveNet (bras grille) : tronc convolutif 3×3 , 8 blocs résiduels de 96 canaux avec un biais de pooling global dans les blocs 2 et 5, une tête de politique plate à 28 673 sorties et une tête de valeur à 3 classes, 1.44 M paramètres. À droite, HiveGraphNet (bras graphe) : couche d'entrée vers 152 canaux, 8 couches relationnelles à passage de messages avec six matrices typées par direction et un biais de pooling global une couche sur trois, une fonction de notation de politique par candidat et une tête de valeur moyenne-et-maximum masquée, 1.47 M paramètres. Les deux alimentent le décodeur d'actions partagé.

Asymétrie de coût mesurée. Ce qui n'a pas pu être apparié est le coût d'exécution de chaque réseau sur la machine d'étude (Apple M1 Pro, 10 cœurs, 16 Go). À une taille de lot de 1, le réseau convolutif s'exécute le plus vite sur l'accélérateur CoreML, à 2.62 ms par évaluation ; le réseau graphe, riche en opérations de collecte (gather), est scindé par CoreML en 15 sous-graphes avec 147 opérateurs pris en charge sur 287 et s'exécute le plus vite sur le fournisseur CPU, à 3.67 ms (9.84 ms sur l'accélérateur), soit un rapport de $\approx 1.4 \times$ au détriment du bras graphe. Le débit d'entraînement au lot 128 est de 274 positions par seconde pour le réseau grille et de 138 pour le réseau graphe. Nous rapportons et imputons cette asymétrie plutôt que de la neutraliser par égalisation, puisque brider un bras fabriquerait une parité qu'aucun utilisateur de l'une ou l'autre représentation ne rencontrerait ; les deux lectures de budget en tiennent compte.

4 Protocole expérimental

Artefacts gelés. Une comparaison sur une seule machine avec une poignée de graines laisse à l'expérimentateur une grande liberté après coup, et nous avons supprimé ces degrés de liberté en prenant chaque engagement avant l'observation qu'il pouvait biaiser, chaque artefact étant gelé sous un hachage de contenu et approuvé par l'auteur. La population d'adversaires et les réglages de recherche de l'évaluation ont été gelés le 9 septembre 2026 ; le protocole (version 1.0, règle de rejet comprise), les 250 ouvertures partagées et la matrice de comparaison avec sa règle de seuil le 10 septembre 2026, après que le pilote eut remesuré les budgets avec les réseaux propres à l'étude, de sorte que le texte gelé ne porte aucune valeur de substitution ; la première exécution de comparaison a démarré le 10 septembre 2026 à 12:42. Le protocole déclare que tout changement ultérieur constitue une nouvelle étude, et aucun n'a été fait.

Conception. Deux bras $\times$ cinq graines $\times$ 10 générations $\times$ 500 parties, à 128/32 simulations par décision avec randomisation du plafond de simulations ; les exécutions sont identiques en tout, hormis l'encodeur d'état et le corps du réseau. Nous lisons une même campagne de deux façons. Sous la lecture à exemples égaux, les deux bras sont comparés à leurs points de contrôle finaux, après s'être entraînés sur le même nombre de parties au même budget de simulation. Sous la lecture à temps mural égal, chaque exécution est lue à son dernier point de contrôle achevé à un seuil à temps mural égal ou avant celui-ci, seuil défini par une règle indépendante des scores dans la matrice pré-enregistrée : la médiane du temps mural d'exécution complète des trois exécutions grille originales, qui donne au bras grille son plein budget par construction et impute au bras graphe son coût réel. La règle a été exécutée le 16 septembre 2026, à partir des seuls chronomètres de la grille et avant qu'aucun nombre inter-bras n'existe : médiane(18.77, 16.73, 19.07) = 18.77 h, temps d'évaluation exclu pour toutes les exécutions de la même manière. Le seuil a sélectionné les indices de génération 9, 9, 8, 9, 9 pour les exécutions grille et 3, 4, 4, 2, 5 pour les exécutions graphe (en comptant à partir de 0) : à temps égal, le bras graphe avait achevé 3–6 de ses dix générations.

Adversaires, ouvertures et évaluation. Hive ne dispose d'aucune échelle publique de réseaux de référence ni d'aucune ancre de jeu parfait, de sorte que nous évaluons contre une population fixe de trois adversaires, tous backends du même moteur : B-RND, uniforme sur les coups légaux ; B-HEU, un argmax glouton à un demi-coup de l'évaluation construite à la main et documentée du moteur, aux poids épinglés par hachage ; et B-MCTS, la même recherche arborescente que les agents appris, avec des a priori uniformes et l'évaluation construite à la main comme valeur de feuille, à 6 400 simulations par décision. Une caractérisation en tournoi toutes rondes le 9 septembre 2026 (100 parties appariées par confrontation) a placé l'heuristique à 100/0/0 et la recherche à 99/1/0 contre l'aléatoire légal, et la recherche à 37.5% contre l'heuristique ; ce dernier ordre n'était pas attendu et n'a délibérément pas été réajusté, puisque le gel interdit d'ajuster un adversaire au vu des résultats. Les parties partent de 250 ouvertures uniques et légales de quatre plis, générées à l'aveugle par une marche aléatoire à graine fixée et gelées sous leur hachage de contenu ; la paire i de chaque affrontement joue la ligne i une fois avec chaque couleur, de sorte que chaque bras, graine, point de contrôle et adversaire fait face à un calendrier d'ouvertures et de couleurs identique. Les évaluations finales et au seuil jouent 100 parties appariées par adversaire à 400 simulations par décision, sans bruit à la racine et avec un meilleur coup déterministe ; les évaluations

intermédiaires aux générations 5 et 8 jouent 20 parties par adversaire et ne servent qu'à la figure de trajectoires. Une vérification de précision le 16 septembre 2026 a trouvé que la dispersion de graine à graine contre l'heuristique (écart-type ≈ 0.056) valait environ deux fois le bruit de parties à 100 parties (erreur-type ≈ 0.03), de sorte que le seul levier était davantage de graines.

Statistiques. La métrique primaire est le score moyen (victoire 1, nulle 0.5, défaite 0) sur les parties non tronquées d'une cellule (graine, adversaire). La graine, une exécution d'entraînement indépendante, est l'unité d'analyse : les parties jouées par un réseau entraîné partagent ses poids et ne sont pas des échantillons indépendants de la méthode. Les moyennes au niveau des graines et les intervalles à 95% sont des bootstrap percentiles à 10 000 rééchantillonnages sur les graines ; pour le contraste graphe-grille, les deux ensembles de graines sont rééchantillonnés indépendamment, puisque les graines d'entraînement ne sont pas appariées entre bras, ce qui est conservateur pour un plan dans lequel les deux bras jouent les mêmes ouvertures. Chaque table montre chaque graine. Les différences s'écrivent graphe moins grille, de sorte qu'un nombre négatif favorise le bras grille.

Troncature. Une partie qui atteint le plafond de 300 plis n'est jamais une nulle. Le score primaire exclut les parties tronquées ; une colonne de sensibilité les compte 0.5 ; deux traitements de bornage comptent chaque partie tronquée du bras testé comme une défaite et comme une victoire ; et la direction du contraste est rapportée sous les quatre traitements. Le traitement « victoire » borne ce que tout plafond plus grand pourrait apporter, ce pour quoi des bornes remplacent une réexécution à plafond plus grand. Le plafond se situe au-delà de la plus longue partie guidée par la recherche observée lors du profilage (202 plis).

Collecte des graines en deux étapes. La matrice pré-enregistre trois graines par bras avec une option à cinq graines « si le budget le permet ». Après l'analyse à trois graines du 19 septembre 2026, l'auteur a approuvé le 26 septembre 2026 les graines 4 et 5 pour les deux bras sous trois pré-engagements écrits avant toute nouvelle exécution : les cinq graines par bras entrent dans l'analyse finale quelle que soit la direction des nouvelles graines ; le seuil à temps mural égal reste à sa valeur déjà calculée ; et la collecte en deux étapes est divulguée. Des graines ajoutées symétriquement sous une règle inchangée sont une décision de puissance statistique plutôt qu'un canal de réglage. Une exécution ne peut être exclue que pour une défaillance de processus, jamais pour un mauvais score ; aucune exécution n'a échoué, n'a été relancée ni n'a été exclue dans aucune campagne.

Ablations. Les règles d'ablation ont été fixées avant les exécutions d'ablation : au plus deux ablations, chacune retirant exactement un composant du bras graphe à pleine parité (graines 1–3, les mêmes budgets, adversaires, ouvertures et évaluation gelés) ; un changement de capacité inhérent au composant retiré est rapporté, jamais compensé ; et là où plus d'un composant diffère, aucun effet n'est attribué au graphe. A1 remplace les six matrices typées par direction par une seule matrice partagée (adjacence naïve, 0.54 M paramètres). A2 retire le biais de pooling global de chaque couche (1.37 M) ; elle a été substituée le 19 septembre 2026 à l'ablation prévue de retrait de l'augmentation, qui n'avait rien à retirer, et ce choix de question, fait après le résultat principal, est divulgué. Les deux ont été approuvées le 20 septembre 2026. Lorsque A1 s'est révélée inentraînable, un supplément A1' a été approuvé le 26 septembre 2026 : arêtes non typées plus un écrêtage global de la norme du gradient à 1.0, le seul changement d'optimiseur de l'étude, étiqueté à deux composants partout où il apparaît.

5 Résultats

Verdict. Sous les deux lectures de budget, le bras graphe ne montre aucun avantage cohérent entre graines contre aucun adversaire, et chaque intervalle sur la différence de score graphe-grille exclut un avantage graphe substantiel : la plus grande borne supérieure sur les six contrastes est +0.028. La règle de rejet pré-enregistrée se déclenche et H1 est rejetée. Tableau 2 donne les moyennes au niveau des graines sous les deux lectures, Tableau 3 les contrastes, et Fig. 3 le score de chaque exécution en fonction de son temps mural d'entraînement.

Tableau 2 : Score moyen au niveau des graines par bras et par adversaire, avec l'intervalle bootstrap percentile à 95% sur les cinq graines (10 000 rééchantillonnages), sous la lecture à exemples égaux (points de contrôle finaux) et la lecture à temps mural égal (dernier point de contrôle dans le seuil de 18.77 h). Le score de chaque graine est la moyenne sur les parties décidées (victoire 1, nulle 0.5, défaite 0) de 100 parties appariées à couleurs échangées par adversaire sur les 250 ouvertures gelées, à 400 simulations par coup ; les parties tronquées au plafond de 300 plis sont exclues.

Lecture	Bras	vs B-RND	vs B-HEU	vs B-MCTS
Exemples égaux	grille	0.981 [0.964, 0.994]	0.129 [0.098, 0.165]	0.124 [0.087, 0.168]
Exemples égaux	graphe	0.812 [0.710, 0.920]	0.065 [0.034, 0.100]	0.115 [0.107, 0.121]
Temps mural égal	grille	0.971 [0.950, 0.991]	0.120 [0.098, 0.142]	0.125 [0.090, 0.168]
Temps mural égal	graphe	0.810 [0.697, 0.922]	0.061 [0.047, 0.081]	0.107 [0.087, 0.130]

Tableau 3 : Le contraste pré-enregistré : différence des scores moyens au niveau des graines (bras graphe moins bras grille), avec l'intervalle bootstrap à 95% sur les graines (cinq graines indépendantes par bras, rééchantillonnées indépendamment, 10 000 rééchantillonnages). Une valeur négative favorise le bras grille.

Adversaire	Exemples égaux : graphe – grille	Temps mural égal : graphe – grille
B-RND (aléatoire légal)	-0.169 [-0.272, -0.062]	-0.161 [-0.278, -0.048]
B-HEU (heuristique)	-0.064 [-0.111, -0.017]	-0.059 [-0.087, -0.029]
B-MCTS (recherche, 6 400 simulations)	-0.009 [-0.055, +0.028]	-0.018 [-0.066, +0.025]

Graine par graine. Sous la lecture à exemples égaux, chaque graine grille obtient au moins 0.949 contre l'adversaire aléatoire légal tandis que les graines graphe s'étendent de 0.688 à 0.981 ; contre l'heuristique, les graines grille s'étendent de 0.080 à 0.190 et les graines graphe de 0.025 à 0.125 ; contre l'adversaire de recherche, les deux bras se recouvrent (0.075–0.210 contre 0.100–0.125). Quatre des six contrastes excluent zéro, tous en faveur du bras grille ; les deux contrastes contre l'adversaire de recherche chevauchent zéro. Le tableau persiste à temps égal. L'analyse à trois graines du 19 septembre 2026 rejetait déjà H1 sous les deux lectures ; l'extension à cinq graines a resserré quatre des six intervalles, absorbé l'unique paire de graines où le graphe faisait mieux observée contre l'heuristique (graine graphe 4 à 0.125 contre graine grille 4 à 0.105) et le meilleur score contre l'adversaire de recherche de toute exécution (graine grille 5 à 0.210), et n'a pas changé le verdict. Contre l'adversaire de recherche, où les deux bras obtiennent un score autour de 0.1, cinq graines par bras ne peuvent pas distinguer les bras.

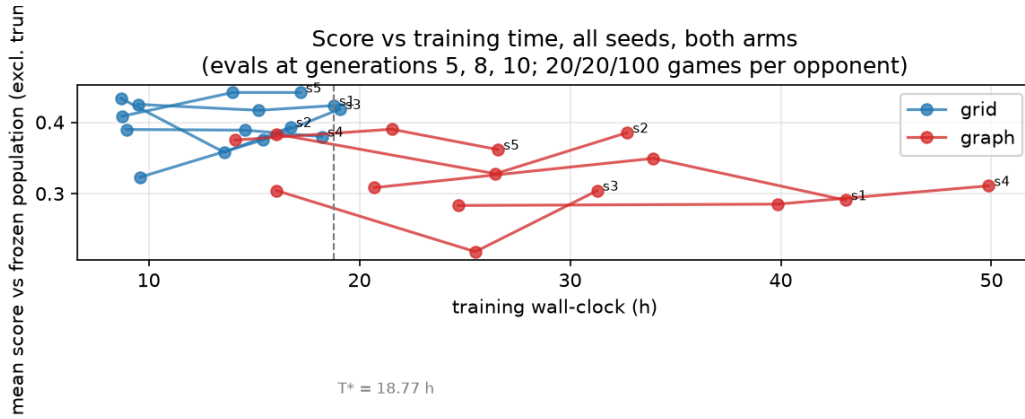


Fig. 3 : Score moyen contre la population gelée (moyenne des trois adversaires, troncatures exclues) en fonction du temps mural d'entraînement cumulé, pour les dix exécutions principales ; évaluations aux générations 5, 8 et 10 (20, 20 et 100 parties par adversaire). Bleu : bras grille ; rouge : bras graphe ; une ligne par graine. La ligne verticale en pointillés marque le seuil à temps mural égal de 18.77 heures.

Le canal de troncature. Le bras graphe a tronqué 57%, 20%, 55%, 44% et 23% de ses parties contre l'adversaire aléatoire au point de contrôle final pour les graines 1 à 5 ; le bras grille a tronqué 0%, 1%, 1%, 11% et 0%. Aucune partie de l'un ou l'autre bras contre l'heuristique ou l'adversaire de recherche n'a atteint le plafond. Chaque graine graphe tronque plus que chaque graine grille (20–57% contre 0–11%). La plus longue partie tronquée du bras graphe contre l'adversaire aléatoire montre le mécanisme : le bras gagne du matériel tôt, puis déplace ses pièces autour d'une reine ennemie partiellement encerclée jusqu'au plafond, en avance tout du long mais sans jamais trouver la séquence forçante qui achève l'encercllement. La pathologie n'est visible que parce que la troncature a été définie comme une issue à part entière avant la première exécution, et elle ne peut pas sauver l'hypothèse : sous le traitement le plus favorable au bras graphe, qui compte chaque partie tronquée comme une victoire pour le bras testé et borne ainsi ce que tout plafond plus grand pourrait apporter, la différence graphe–grille contre l'adversaire aléatoire (calculée à partir des enregistrements par partie des trois graines originales) est encore de -0.072 à exemples égaux et de -0.058 à temps égal, et la direction du contraste est inchangée sous chaque traitement. Le score du bras graphe contre l'adversaire aléatoire repose bel et bien sur moins de parties décidées (entre 43 et 80 par graine), ce qui élargit son intervalle.

Coût. Tableau 4 résume ce que chaque bras coûte sur la machine d'étude. L'évaluation unitaire du bras graphe est $1.4\times$ plus lente à son meilleur fournisseur, son débit d'entraînement est la moitié de celui du bras grille, et une exécution complète lui coûte $2.0\times$ le temps mural (36.7 h contre 18.0 h en moyenne ; les cinq graines grille ont pris 18.77, 16.73, 19.07, 18.23 et 17.19 heures et les cinq graines graphe 43.07, 32.71, 31.28, 49.86 et 26.57 heures). Le score contre la population (moyenne, sur les trois adversaires, du score moyen au niveau des graines) est de 0.411 pour le bras grille et de 0.331 pour le

bras graphe sous la lecture à exemples égaux, de 0.405 et 0.326 sous la lecture à temps mural égal : le même ordre et presque le même écart, 0.081 et 0.079 en faveur du bras grille. Le budget a été lu deux fois au cas où une représentation apprendrait davantage par exemple mais moins par heure ; ici les deux lectures concordent, et la lecture à temps égal accentue un déficit que la lecture par exemple montre déjà. Les dix exécutions principales ont consommé 273.5 h de temps mural d'entraînement (grille 90.0 h, graphe 183.5 h), le tout sur un seul ordinateur portable, sans calcul payant.

Tableau 4 : Résumé des coûts et des scores par bras. Le temps mural d'entraînement couvre la génération en auto-jeu et l'entraînement pour les dix générations d'une exécution, parties d'évaluation exclues, moyenné sur les cinq graines ; les chiffres d'inférence et de débit sont des mesures de référence du 10 septembre 2026 sur la machine d'étude (Apple M1 Pro, 10 cœurs, 16 Go). Score contre la population = moyenne, sur les trois adversaires gelés, du score moyen au niveau des graines, troncatures exclues.

Métrique	Bras grille	Bras graphe
Paramètres	1.44 M	1.47 M (+1.5%)
Inférence au meilleur fournisseur, lot 1	2.62 ms (accélérateur neuronal)	3.67 ms (CPU)
Temps mural d'entraînement par exécution, moyenne de 5 graines	18.0 h	36.7 h (2.0×)
Débit d'entraînement (lot 128, avant + arrière)	274 pos/s	138 pos/s
Score contre la population, lecture à exemples égaux	0.411	0.331
Score contre la population, lecture à temps mural égal (seuil 18.77 h)	0.405	0.326

5.1 Ablations

Chaque ablation retire un composant du bras graphe et rien d'autre, avec trois graines par cellule ; les différences sont variante moins bras graphe complet, en moyennes au niveau des graines avec intervalles bootstrap sur les graines (10 000 rééchantillonnages, ensembles de graines indépendants).

Avec une seule matrice de messages partagée à la place de six matrices typées (A1), l'entraînement a divergé vers des valeurs non finies à la génération 0 pour les trois graines, et chaque génération ultérieure a joué en auto-jeu et s'est entraînée sur des sorties non finies. Le symptôme était statistique plutôt que numérique : les évaluations finales des trois graines étaient identiques jusque dans le nombre de parties, ce que des entraînements indépendants à graines distinctes ne peuvent produire. Les tables d'évaluation de la cellule ne sont donc pas des mesures de force, et le résultat se lit « entraînement divergé (3/3 graines) ». L'explication que nous proposons est un argument raisonné plutôt qu'une décomposition mesurée : une seule matrice partagée reçoit le gradient sommé de six termes de voisinage, soit environ six fois l'échelle de gradient par matrice de la variante typée, à taux d'apprentissage et à momentum identiques. Aucun balayage du taux d'apprentissage n'a été exécuté, parce qu'il aurait fait de A1 une différence à deux composants.

Retirer le biais de pooling global (A2) a changé le score de -0.001 $[-0.170, +0.165]$ contre l'adversaire aléatoire, de -0.007 $[-0.043, +0.030]$ contre l'heuristique et de -0.013 $[-0.043, +0.013]$ contre l'adversaire de recherche. Chaque intervalle chevauche zéro, et la pathologie de conversion contre l'adversaire aléatoire est présente dans les deux variantes

(taux de troncature entre 17% et 65% selon les graines). À cette échelle, le biais de pooling global est dispensable. Le résultat nul est borné par des intervalles de largeur ± 0.17 contre l'adversaire aléatoire, où un petit effet pourrait se cacher.

La variante à adjacence naïve stabilisée (A1', relations non typées plus écrêtage de gradient à 1.0, un changement à deux composants) s'est entraînée de façon finie pour les trois graines et a obtenu des scores dans la bande du bras graphe complet, les moyennes à cinq graines du bras complet servant de référence : -0.066 $[-0.254, +0.130]$, $+0.032$ $[-0.011, +0.078]$ et $+0.008$ $[-0.040, +0.063]$ contre les trois adversaires, chaque intervalle chevauchant zéro. Chaque nombre de cette cellule est confondu par l'écrêtage par construction et n'est jamais attribué au seul typage des arêtes. Combiné avec A1, le plus que l'on puisse dire est qu'à cette échelle la contribution mesurable des relations typées par direction réside dans la stabilité d'optimisation ; aucune contribution à la force au-delà de celle-ci n'est détectable.

6 Discussion

Lire le résultat. Le déficit est modelé par l'adversaire : le plus grand contre l'adversaire aléatoire légal, net contre l'heuristique, absent contre l'adversaire de recherche. Un bras graphe qui serait simplement plus faible partout montrerait un déficit uniforme ; un déficit concentré là où les parties se décident en achevant un encerclement auquel rien ne résiste désigne la conversion tactique locale plutôt que le jugement positionnel. Les métriques d'entraînement concordent : les deux bras ajustent leurs données d'auto-jeu tout au long des dix générations, et l'exactitude de la valeur du bras graphe sur les échantillons non tronqués est comparable à celle du bras grille, tandis que son argmax de politique (top-1) reste en retrait. L'écart n'est donc pas un simple échec d'optimisation du bras graphe complet ; il sépare deux réseaux entraînés, dont l'un apprend une évaluation utilisable et une politique plus faible.

Explications plausibles et démontrées. La géométrie est plausible mais non démontrée : la tactique de Hive est dominée par la géométrie d'encerclement à courte portée, le régime dans lequel Keller et al. (2023) ont trouvé les réseaux convolutifs plus forts sur Hex ; une convolution 3×3 sur le cadre déplié voit tout le voisinage d'une cellule sans coût d'apprentissage, alors que le réseau à passage de messages doit composer les mêmes motifs à partir de six relations typées, un saut à la fois, et aucune ablation de cette étude n'isole la composition du champ récepteur. La capacité est contrôlée : les réseaux diffèrent de +1.5%, en faveur du bras graphe. Le décodeur est contrôlé : les deux bras notent l'ensemble légal identique avec un masquage, une normalisation, des cibles et un départage identiques. La vitesse est démontrée pour la seconde lecture seulement : les générations du bras graphe coûtent environ deux fois plus de temps mural, de sorte qu'à temps égal il achève moins de la moitié de ses générations, ce qui élargit l'écart dans la lecture à temps mural égal et ne joue aucun rôle dans la lecture à exemples égaux, où le déficit existe déjà. L'optimisation est partiellement démontrée : les ablations montrent que l'optimisation du bras graphe est fragile d'une manière précise (elle dépend des relations typées pour sa stabilité), tandis que le bras complet ajuste ses données pour chaque graine ; la question de savoir si un taux d'apprentissage ou une normalisation

différents auraient favorisé le bras graphe n'a pas été testée, parce qu'aucun des deux bras n'a reçu de réglage.

Rapport aux travaux antérieurs. Le résultat ne contredit ni les constats positifs de Rigaux et Kashima (2024) sur les graphes aux échecs, ni l'avantage à longue portée que Keller et al. (2023) ont mesuré sur Hex ; il borne là où leur optimisme se transfère. Le résultat aux échecs repose sur une seule exécution d'entraînement par modèle, un appariement de capacité lâche et des décodeurs propres à chaque bras ; le résultat sur Hex a été obtenu sous un apprenant fondé sur la valeur avec une construction de graphe propre au jeu, et a trouvé le bras convolutif plus affûté sur les motifs locaux, le régime qui décide des parties de Hive aux petits budgets. Notre rejet est cohérent entre graines sur cinq exécutions par bras sous deux lectures de budget pré-enregistrées, à capacité appariée et avec le décodeur partagé. Lues ensemble, les trois études suggèrent que le signe de la comparaison dépend de la portée tactique du jeu et du standard de preuve plutôt que du paradigme des graphes en tant que tel. AZ-Hive (de Goede et al., 2022) avait montré que l'encodage change l'apprentissage au sein de la famille grille ; nous étendons le constat d'une famille à l'autre et trouvons le côté grille en avance.

Menaces à la validité. Interne : le moteur est validé indépendamment de l'apprentissage, les deux encodeurs sont épinglés à l'octet près, et les bras partagent le décodeur, les enregistrements, les fonctions de perte, les budgets, la recherche et l'évaluation gelée ; l'architecture graphe n'en est pas moins un point de l'espace de conception, et rien n'est affirmé au-delà de celui-ci. Des défauts d'outillage ont été trouvés pendant l'étude (un chemin du lanceur de matchs qui perdait des enregistrements, une boucle d'entraînement qui laissait passer silencieusement des valeurs non finies, et un générateur de résultats qui moyennait le temps mural de cinq exécutions sur trois) ; chacun a été attrapé par la vérification mécanique, corrigé et audité, et aucun n'a touché un score, un intervalle ni une affirmation. Mesure : les scores sont relatifs à trois adversaires fixes couvrant une force du plancher au niveau intermédiaire ; tous les bras entraînés perdent encore lourdement contre les deux adversaires forts après dix générations, de sorte que les différences contre l'adversaire de recherche ne peuvent pas être résolues. Statistique : cinq graines par bras bornent les statistiques ; le rejet repose sur la cohérence entre graines à travers deux adversaires et deux lectures, sur quatre des six intervalles excluant zéro et sur les bornes du plafond, et non sur un seul intervalle. Externe : une variante (Hive de base, sans extensions), une machine, un petit budget (entre 16.7 et 19.1 heures de temps mural d'entraînement par exécution grille et entre 26.6 et 49.9 heures par exécution graphe), auto-jeu en régime précoce tout du long. La politique de réglage nul est symétrique mais peut ne pas être neutre : les réseaux de graphes sont communément plus sensibles aux valeurs par défaut de l'optimiseur que les réseaux convolutifs résiduels, et la première ablation a montré que cette famille est fragile sous ces valeurs par défaut ; un bras graphe réglé pourrait réduire l'écart, au prix de la rupture de la symétrie de la comparaison.

La propriété de réinitialisation comme limite. Le réseau de chaque génération est entraîné à partir d'une initialisation fraîche à graine fixée sur les seuls enregistrements de cette génération, de sorte que chaque réseau voit les positions de 500 parties, un petit ensemble d'entraînement au regard des standards de l'apprentissage par auto-jeu ; la force absolue atteinte après dix générations (près du plafond contre l'adversaire aléatoire, autour de 0.1 contre les deux adversaires forts) doit être lue à la lumière de ce fait. Pour la comparaison, la propriété est neutre : elle est identique dans les deux bras, et les

deux encodeurs voient exactement les mêmes enregistrements. Pour la généralisation du résultat, c'est une limite qui s'ajoute à celles ci-dessus : un pipeline qui accumulerait les données et reporterait les poids atteindrait un régime différent, dans lequel l'ordre des bras est une question ouverte.

Deux suites. La première vise la pathologie de conversion, que les données isolent comme un défaut de politique plutôt qu'un défaut de valeur : un remède au moment de la recherche, appliqué à l'identique aux deux bras sous la même évaluation gelée, tel qu'un budget d'évaluation plus profond dans les positions que la tête de valeur juge déjà gagnées, ou des cibles d'entraînement auxiliaires pour les coups forçants. La seconde concerne le constat de stabilité : une étude contrôlée des choix de normalisation et d'échelle de gradient pour des couches de graphe à relations partagées découplerait l'entraînabilité du contenu représentationnel et dirait si l'adjacence naïve, correctement stabilisée, est suffisante pour Hive.

7 Conclusion

Dans le périmètre testé (Hive en jeu de base, un réseau relationnel à passage de messages à capacité appariée contre un réseau convolutif en grille, dix générations d'auto-jeu à petit budget, cinq graines indépendantes par bras, une population gelée de trois adversaires et 250 ouvertures gelées), la réponse à la question de recherche principale est non : la représentation en graphe n'a pas appris une meilleure politique que la représentation en grille, ni à exemples d'entraînement égaux ni à temps mural égal sur la même machine, et la règle de rejet pré-enregistrée s'est déclenchée exactement telle qu'elle avait été gelée. Le déficit est le plus grand là où Hive est le plus tactique (-0.169 et -0.161 contre l'adversaire aléatoire légal sous les deux lectures), net contre l'heuristique (-0.064 et -0.059) et absent contre l'adversaire de recherche (-0.009 et -0.018). Le bras graphe paie environ deux fois le temps mural par exécution, et son mode de défaillance caractéristique consiste à gagner du matériel sans convertir le gain, ce qui se manifeste en ce que 20–57% de ses parties contre l'adversaire aléatoire se terminent au plafond de coups. De ses deux composants distinctifs, les relations typées par direction sont nécessaires à l'optimisation elle-même, tandis que le biais de pooling global est dispensable à cette échelle. Là où les tactiques d'encercllement à courte portée décident des parties et où les budgets sont petits, les artefacts de cadre d'un encodage en grille se révèlent moins coûteux que l'absence de cadre.

Reproductibilité et disponibilité

L'ensemble publié est un seul dépôt contenant le moteur Rust, le code Python d'entraînement et d'export, les configurations gelées (matrice de comparaison, réglages d'évaluation, configurations des adversaires avec les poids de l'heuristique épinglés par hachage, spécifications d'ablation), les 250 ouvertures gelées et le manifeste de la population avec ses hachages, les tables de résultats, et les enregistrements par exécution : un fichier par point de contrôle et par adversaire avec une ligne par partie, le temps

mural par génération, les manifestes et les segments (shards) d'auto-jeu, et le point de contrôle de chaque génération. Les moteurs de référence tiers utilisés dans la validation des règles ne sont pas livrés, puisqu'aucun nombre de l'étude n'en dérive. L'accès et l'étiquette de publication sont fixés au moment de la diffusion ; au moment de la rédaction, rien n'a quitté la machine d'étude.

Un scénario minimal en environnement vierge, exécuté le 9 octobre 2026, effectuée, à partir d'un clone propre et des enregistrements livrés, la plus petite vérification de bout en bout qui touche chaque maillon de la chaîne : construire le moteur et l'arène ; rejouer une partie enregistrée de manière déterministe (le point de contrôle final de la graine 2 du bras grille contre l'adversaire heuristique sur la ligne d'ouverture 2, le bras jouant les Noirs, perdue en 19 plis) et vérifier que sa ligne de résultat est égale à la ligne livrée ; et régénérer les tables des deux lectures à partir des enregistrements livrés, identiques à l'octet près à celles publiées. Il a réussi. Le scénario ne vérifie pas l'entraînement ; la reproduction complète répète les dix exécutions d'entraînement (16.7–19.1 h par exécution grille et 26.6–49.9 h par exécution graphe sur la machine d'étude), les évaluations au seuil et les ablations, et régénère chaque table et chaque figure à partir des enregistrements bruts par partie.

Déclaration d'assistance par un système d'intelligence artificielle. Cette étude a été exécutée selon une méthodologie « l'humain comme chercheur principal », dans laquelle un assistant de recherche IA (Claude, Anthropic) a implémenté le code, mené les campagnes de calcul, tenu les journaux d'expériences et rédigé le texte, sous un système de six portes de décision réservant chaque engagement scientifique à l'auteur humain : le gel du protocole, des adversaires et des ouvertures, chaque dépense de calcul, tout ce qui est rendu public et la suppression de toute donnée. L'auteur humain a choisi la question, approuvé par écrit chaque décision soumise à une porte, vérifié les conclusions et assume chaque affirmation.

Références

- Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., & Bellemare, M. G. (2021). Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems* (NeurIPS 2021) (peer-reviewed; Outstanding Paper Award). arXiv:2108.13264 (v4, 5 January 2022). <https://doi.org/10.48550/arXiv.2108.13264>
- Ben-Assayag, S., & El-Yaniv, R. (2021). Train on small, play the large: Scaling up board games with AlphaZero and GNN. arXiv preprint arXiv:2107.08387v1 (18 July 2021) (no peer-reviewed version found as of 26 September 2026). <https://doi.org/10.48550/arXiv.2107.08387>
- de Goede, D., Kampert, D., & Varbanescu, A. L. (2022). The cost of reinforcement learning for game engines: The AZ-Hive case-study. In *Proceedings of the 13th ACM/SPEC International Conference on Performance Engineering* (ICPE 2022, Beijing), pp. 145–152. <https://doi.org/10.1145/3489525.3511685>
- Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems* (NIPS 2017) (peer-reviewed). arXiv:1706.02216 (v4, 10 September 2018). <https://doi.org/10.48550/arXiv.1706.02216>

Jones, A. L. (2021). Scaling scaling laws with board games. arXiv preprint arXiv:2104.03113v2 (15 April 2021) (not peer-reviewed). <https://doi.org/10.48550/arXiv.2104.03113>

Kampert, D., Varbanescu, A.-L., Müller-Brockhausen, M., & Plaat, A. (2021). Mimicking the human approach in the game of Hive. In *2021 IEEE Symposium Series on Computational Intelligence* (SSCI 2021, Orlando). IEEE Xplore document 9659999. <https://ieeexplore.ieee.org/document/9659999/>. Preprint circulated as «Better AI for Hive: Mimicking human game-play strategies».

Keller, Y., Blüml, J., Sudhakaran, G., & Kersting, K. (2023). From images to connections: Can DQN with GNNs learn the strategic game of Hex? arXiv preprint arXiv:2311.13414 (22 November 2023) (not peer-reviewed; OpenReview submission dYaeDrazj5). <https://arxiv.org/abs/2311.13414>

Rigaux, T., & Kashima, H. (2024). Enhancing chess reinforcement learning with graph representation. In *Advances in Neural Information Processing Systems 37* (NeurIPS 2024, main conference track). <https://doi.org/10.52202/079017-0006>. Preprint: arXiv:2410.23753v1 (31 October 2024), <https://doi.org/10.48550/arXiv.2410.23753>

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144 (peer-reviewed). <https://doi.org/10.1126/science.aar6404>. Preprint (2017): Mastering chess and shogi by self-play with a general reinforcement learning algorithm, arXiv:1712.01815v1 (5 December 2017), <https://doi.org/10.48550/arXiv.1712.01815>

Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. In *Advances in Neural Information Processing Systems 28* (NIPS 2015) (peer-reviewed). arXiv:1506.03134 (v2, 2 January 2017). <https://arxiv.org/abs/1506.03134>

Wu, D. J. (2020). Accelerating self-play learning in Go. arXiv preprint arXiv:1902.10565v5 (9 November 2020); presented at the AAAI-20 Workshop on Reinforcement Learning in Games (not a full peer-reviewed proceedings paper). <https://doi.org/10.48550/arXiv.1902.10565>

Webographie.

edre (GitHub user). *nokamute*: Hive engine in Rust, with a Universal Hive Protocol conformance tester and a built-in match runner [source-code repository]. GitHub. <https://github.com/edre/nokamute>. Version 1.0.3 used as a reference engine. Consulté en juillet 2026 (conception du moteur) et le 9 septembre 2026 (campagne de validation).

jonthysell (GitHub user). *Mzinga*: reference Hive engine and project wiki, including the Universal Hive Protocol specification and the perft tables [source-code repository and wiki]. GitHub. <https://github.com/jonthysell/Mzinga>; perft tables: <https://github.com/jonthysell/Mzinga/wiki/Perft>. Release MzingaEngine v0.16.0 used as a reference engine. Consulté en juillet 2026 (conception du moteur) et le 9 septembre 2026 (campagne de validation).

World Hive Tournaments. *Rules of Hive: Rules FAQ* [web page]. <https://www.worldhivetournaments.com/rules-of-hive/>. Consulté le 9 septembre 2026.

Yianni, J. (2010). *Hive rules* [publisher's rules sheet]. Gen42 Games. <https://www.gen42.com/wp-content/uploads/Hive-rules.pdf>. Consulté le 9 septembre 2026.

v2.0-draft, 2026-10-09